

TRADUTOR DE LIBRAS PARA ÁUDIO COM INTELIGÊNCIA ARTIFICIAL

Relatório Final Atualizado – Projeto Prático com Desenvolvimento de Protótipo

Júlia Gabriela Gomes da Silva

Lara Ewellen de Carvalho Rocha

Luiza Lima de Sá

Maria Eduarda Nunes Bezerra

Brasília

2026

RESUMO

Este trabalho apresenta a versão atualizada do protótipo funcional de tradução de Língua Brasileira de Sinais (Libras) para texto e áudio, utilizando visão computacional e aprendizado de máquina. O sistema captura imagens por webcam, extrai landmarks com MediaPipe e classifica sinais por meio de uma rede neural temporal composta por camadas Conv1D, LSTM bidirecional e mecanismo de atenção. A versão atual do modelo foi treinada com 13 classes de sinais isolados, incluindo as novas classes Eu e Quero, e atingiu acurácia de 64.35% no conjunto de teste, com acurácia balanceada de 59.63%. A interface web integra síntese de voz via Web Speech API, permitindo a verbalização dos sinais reconhecidos em português do Brasil.

Palavras-chave: Libras. Visão computacional. Aprendizado de máquina. Acessibilidade. Síntese de voz.

SUMÁRIO

- 1 Objetivo principal
- 2 Descrição do problema e relevância
- 3 Escopo do protótipo
- 4 Organização do projeto
- 5 Tecnologias utilizadas e justificativa
- 6 Base de dados
- 7 Pré-processamento e extração de características
- 8 Modelo de inteligência artificial
- 9 Funcionamento do pipeline
- 10 Regras de estabilização em tempo real
- 11 Interface e síntese de voz
- 12 Resultados e métricas
- 13 Dificuldades encontradas
- 14 Análise crítica
- 15 Trabalhos futuros
- 16 Como executar o projeto
- 17 Conclusão
- 18 Referências
- Anexos

1 OBJETIVO PRINCIPAL

O objetivo deste projeto é desenvolver um protótipo funcional de um sistema baseado em Inteligência Artificial capaz de reconhecer sinais de Libras a partir da câmera de um computador e converter o resultado em texto e áudio. A proposta busca criar uma ponte de comunicação entre pessoas que utilizam Libras e pessoas que não dominam a língua.

O protótipo reconhece sinais isolados em tempo real. O sistema captura imagens da webcam, extrai pontos de referência do corpo, das mãos e do rosto, organiza esses dados como uma sequência temporal, classifica o sinal por meio de uma rede neural treinada e apresenta o resultado na interface web.

2 DESCRIÇÃO DO PROBLEMA E RELEVÂNCIA

A comunicação entre pessoas surdas que utilizam Libras e pessoas ouvintes que não conhecem a língua ainda depende, em muitos contextos, da presença de intérpretes ou de alternativas improvisadas de comunicação.

O uso de computadores e smartphones com câmera torna possível propor soluções de apoio mais acessíveis. A ideia central do projeto é usar a câmera como sensor, transformar o movimento do usuário em dados numéricos e aplicar um modelo de IA para reconhecer os sinais.

3 ESCOPO DO PROTÓTIPO

Funcionalidade	Implementação no projeto
Captura de vídeo	Interface web acessa a webcam pelo navegador usando getUserMedia.
Rastreamento corporal	Backend usa MediaPipe para extrair landmarks de mãos, pose corporal e rosto.
Classificação dos sinais	Modelo TensorFlow/Keras classifica sequências de 20 frames.
Geração de texto	Interface mostra sinal reconhecido, confiança, histórico e top predições.
Síntese de voz	Frontend usa Web Speech API para falar o sinal reconhecido.
Feedback humano	Interface registra acertos e correções em novas amostras rotuladas.

O vocabulário atual do modelo possui 13 classes.

Classe	Papel no sistema
Aceitar	Sinal reconhecido
Banheiro	Sinal reconhecido
Bebida	Sinal reconhecido
Calmo	Sinal reconhecido
Casa	Sinal reconhecido
Desconhecido	Classe de rejeição
Eu	Sinal reconhecido
Livro	Sinal reconhecido
Nome	Sinal reconhecido
Obrigado	Sinal reconhecido

Classe	Papel no sistema
Oi	Sinal reconhecido
Quero	Sinal reconhecido
Sortudo	Sinal reconhecido

4 ORGANIZAÇÃO DO PROJETO

Arquivo/Pasta	Função
app.py	Backend Flask responsável por receber frames, extrair features e retornar predições.
etapa2_preprocessamento.py	Extrai landmarks dos vídeos e gera data/landmarks.csv.
etapa3_treinamento.py	Treina o modelo, calcula métricas e salva artefatos.
front/	Interface web com HTML, CSS e JavaScript.
models/	Modelo treinado, encoder de labels e thresholds.
reports/	Métricas, matriz de confusão e gráficos de treinamento.

5 TECNOLOGIAS UTILIZADAS E JUSTIFICATIVA

Tecnologia	Uso	Justificativa
Python 3.11	Backend, processamento e treinamento	É o sistema amplo para IA e visão computacional.
Flask	Servidor web	API simples para receber frames e retornar predições.
OpenCV	Manipulação de imagens	Decodifica frames e prepara imagens.
MediaPipe	Extração de landmarks	Detecta mãos, pose e rosto em tempo real.
TensorFlow/Keras	Modelo neural	Implementa Conv1D, LSTM e atenção temporal.
Scikit-learn	Métricas e labels	Gera acurácia, relatório e matriz de confusão.
Web Speech API	Síntese de voz	Fala os sinais reconhecidos no navegador.

A escolha do MediaPipe foi importante porque ele transforma imagens em landmarks numéricos. Isso reduz a complexidade do problema, pois o modelo aprende a partir de coordenadas de mãos, corpo e rosto em vez de pixels brutos.

6 BASE DE DADOS

Os vídeos ficam em videos/data/ e seguem o padrão NomeDoSinal_ArticuladorN.mp4.

Classe	Vídeos
Aceitar	20
Banheiro	20
Bebida	22
Calmo	20
Casa	20
Desconhecido	63

Classe	Vídeos
Eu	27
Livro	20
Nome	20
Obrigado	20
Oi	20
Quero	28
Sortudo	20

A versão atual inclui as classes Eu e Quero, ampliando o vocabulário em relação à versão anterior.

7 PRÉ-PROCESSAMENTO E EXTRAÇÃO DE CARACTERÍSTICAS

Cada frame é convertido em um vetor com 288 features, combinando pose corporal, mão esquerda, mão direita e pontos selecionados do rosto.

Grupo de landmarks	Quantidade	Valores por ponto	Total
Pose corporal	33 pontos	x, y, z, visibilidade	132
Mão esquerda	21 pontos	x, y, z	63
Mão direita	21 pontos	x, y, z	63
Rosto selecionado	10 pontos	x, y, z	30
Total	-	-	288

O sistema trabalha com sequências de 20 frames, formando uma entrada de 20 x 288 para o modelo.

7.1 Normalização dos landmarks

Os landmarks são normalizados para reduzir variações de enquadramento, distância da câmera e tamanho da pessoa.

7.2 Janelas de vídeo

O pré-processamento divide os vídeos em janelas de 20 frames e avalia critérios como detecção de mãos, movimento e taxa de aceitação.

8 MODELO DE INTELIGÊNCIA ARTIFICIAL

O projeto usa aprendizado supervisionado: cada sequência de landmarks é associada a um rótulo, como Oi, Casa, Eu ou Quero.

8.1 Arquitetura do modelo

A arquitetura combina Conv1D, Batch Normalization, Dropout, duas camadas LSTM bidirecionais, mecanismo de atenção temporal, camada densa e saída softmax.

8.2 Data augmentation

Conjunto	Amostras
Treino original	1464
Validação	293
Teste	432
Treino após augmentation	10539

As augmentações incluem perturbações controladas nos landmarks, variações de escala, deslocamento, velocidade e dropout de pontos.

9 FUNCIONAMENTO DO PIPELINE

O pipeline segue: câmera ou vídeo, captura de frames, extração de landmarks com MediaPipe, normalização, sequência de 20 frames, classificação pelo modelo temporal, filtros de estabilidade, texto e síntese de voz.

Em tempo real, o navegador captura frames, envia para /predict, o Flask decodifica com OpenCV, o MediaPipe extrai landmarks e o modelo prediz probabilidades.

10 REGRAS DE ESTABILIZAÇÃO EM TEMPO REAL

Recurso	Descrição
Threshold por classe	Cada classe possui um limite mínimo de confiança.
Votação	A predição precisa se repetir em uma janela recente.
Margem	A diferença entre primeira e segunda classe precisa ser suficiente.
Movimento mínimo	Evita aceitar sequências paradas.
Debounce	Evita repetir o mesmo sinal várias vezes.
Classe Desconhecido	Ajuda a rejeitar gestos fora do vocabulário.

Classe	Threshold
Aceitar	0.75
Banheiro	0.55
Bebida	0.84
Calmo	0.55
Casa	0.92
Desconhecido	0.59
Eu	0.55
Livro	0.75
Nome	0.94
Obrigado	0.55
Oi	0.75
Quero	0.73
Sortudo	0.77

11 INTERFACE E SÍNTESE DE VOZ

A interface web permite iniciar a câmera, visualizar predições, acompanhar top classes, montar frase simples, falar o sinal por síntese de voz e registrar feedback. A síntese usa Web Speech API com idioma pt-BR.

12 RESULTADOS E MÉTRICAS

Métrica	Resultado
Acurácia final	64.35%
Acurácia balanceada	59.63%
Amostras de teste	432
Classes avaliadas	13

Classe	Precisão	Recall	F1-score	Suporte
Aceitar	0.36	0.69	0.47	29
Banheiro	0.86	0.61	0.71	41
Bebida	0.85	0.93	0.89	43
Calmo	0.85	0.59	0.70	49
Casa	0.38	0.58	0.46	26
Desconhecido	0.58	0.60	0.59	43
Eu	0.86	0.65	0.74	37
Livro	0.47	0.57	0.52	28
Nome	0.31	0.36	0.33	11
Obrigado	0.44	0.19	0.27	21
Oi	0.47	0.42	0.44	19
Quero	0.86	0.83	0.84	52
Sortudo	0.77	0.73	0.75	33

A acurácia de 64.35% indica que o protótipo reconhece parte relevante dos sinais, mas ainda não tem robustez suficiente para uso real amplo.

As classes com melhor desempenho foram Bebida, Quero, Sortudo, Eu, Banheiro e Calmo. As mais problemáticas foram Obrigado, Nome, Oi, Casa e Aceitar.

Quantidade	Classe real	Classe prevista
10	Eu	Desconhecido
7	Quero	Casa
7	Desconhecido	Aceitar
6	Obrigado	Casa
6	Livro	Aceitar

Quantidade	Classe real	Classe prevista
6	Casa	Livro
6	Calmo	Aceitar

13 DIFICULDADES ENCONTRADAS

Dificuldade	Impacto
Dataset limitado	Poucos exemplos por classe dificultam a generalização.
Variação entre articuladores	Pessoas diferentes executam o mesmo sinal com diferenças.
Iluminação e enquadramento	Podem afetar a detecção dos landmarks.
Sinais parecidos	Alguns gestos compartilham trajetórias ou posições semelhantes.
Reconhecimento em tempo real	Predições quadro a quadro podem oscilar.
Classe Desconhecido	É difícil representar todos os movimentos fora do vocabulário.

14 ANÁLISE CRÍTICA

O protótipo atende ao objetivo de demonstrar um pipeline completo de IA para tradução de sinais isolados de Libras para texto e áudio.

Ao mesmo tempo, a acurácia de 64.35% mostra que o sistema ainda está em fase experimental e não deve ser visto como ferramenta pronta para uso real de acessibilidade sem supervisão.

Libras envolve estrutura gramatical própria, expressões faciais, contexto, direção do olhar, configuração das mãos, orientação, ponto de articulação e movimento. O protótipo ainda trabalha com sinais isolados.

15 TRABALHOS FUTUROS

Melhoria	Benefício esperado
Aumentar o dataset	Melhorar acurácia e generalização.
Gravar mais pessoas	Reduzir dependência de um estilo específico.
Variar ambiente e iluminação	Tornar o modelo mais robusto.
Melhorar a classe Desconhecido	Reduzir falsas traduções.
Expandir vocabulário	Reconhecer mais sinais.
Segmentar início e fim	Melhorar reconhecimento em fluxo contínuo.
Testar Transformer ou TCN	Comparar arquiteturas modernas.
Usar modelos de linguagem	Permitir frases mais naturais.
Exportar para TensorFlow Lite	Facilitar execução em dispositivos móveis.

16 COMO EXECUTAR O PROJETO

Preparar Python 3.11, criar e ativar a .venv, instalar requirements.txt e executar: validar_dataset.py, etapa2_preprocessamento.py, etapa3_treinamento.py e app.py. Depois, acessar

<http://localhost:5000>.

17 CONCLUSÃO

O projeto desenvolveu um protótipo funcional de Tradutor de Libras com IA. A versão atual trabalha com 13 classes e atingiu 64.35% de acurácia no conjunto de teste e 59.63% de acurácia balanceada.

A abordagem é promissora, mas ainda exige ampliação da base, testes com diferentes usuários, melhoria da rejeição de gestos desconhecidos e evolução para frases completas.

18 REFERÊNCIAS

IA Libras. Disponível em: <https://ialibras.com.br/>

Hand Talk. Disponível em: <https://www.handtalk.me/br/>

MediaPipe. Documentação oficial. Disponível em: <https://developers.google.com/mediapipe>

TensorFlow. Documentação oficial. Disponível em: <https://www.tensorflow.org/>

Keras. Documentação oficial. Disponível em: <https://keras.io/>

Scikit-learn. Documentação oficial. Disponível em: <https://scikit-learn.org/>

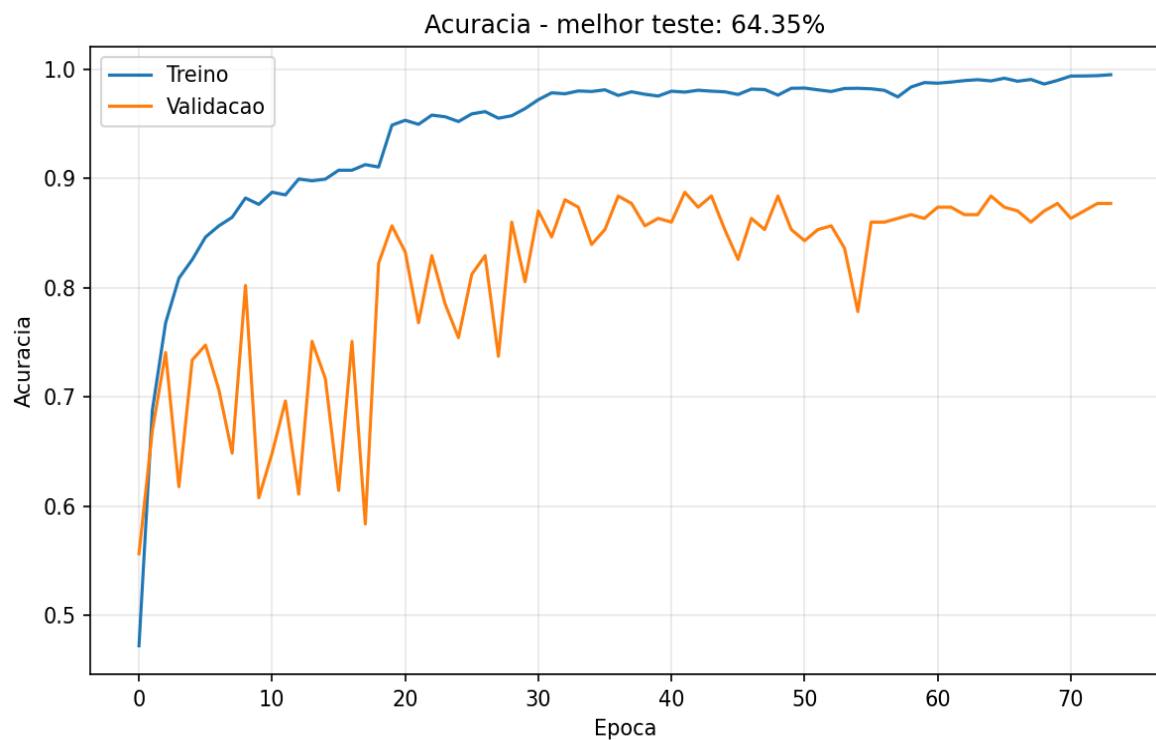
Flask. Documentação oficial. Disponível em: <https://flask.palletsprojects.com/>

MDN Web Docs. Web Speech API. Disponível em:
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

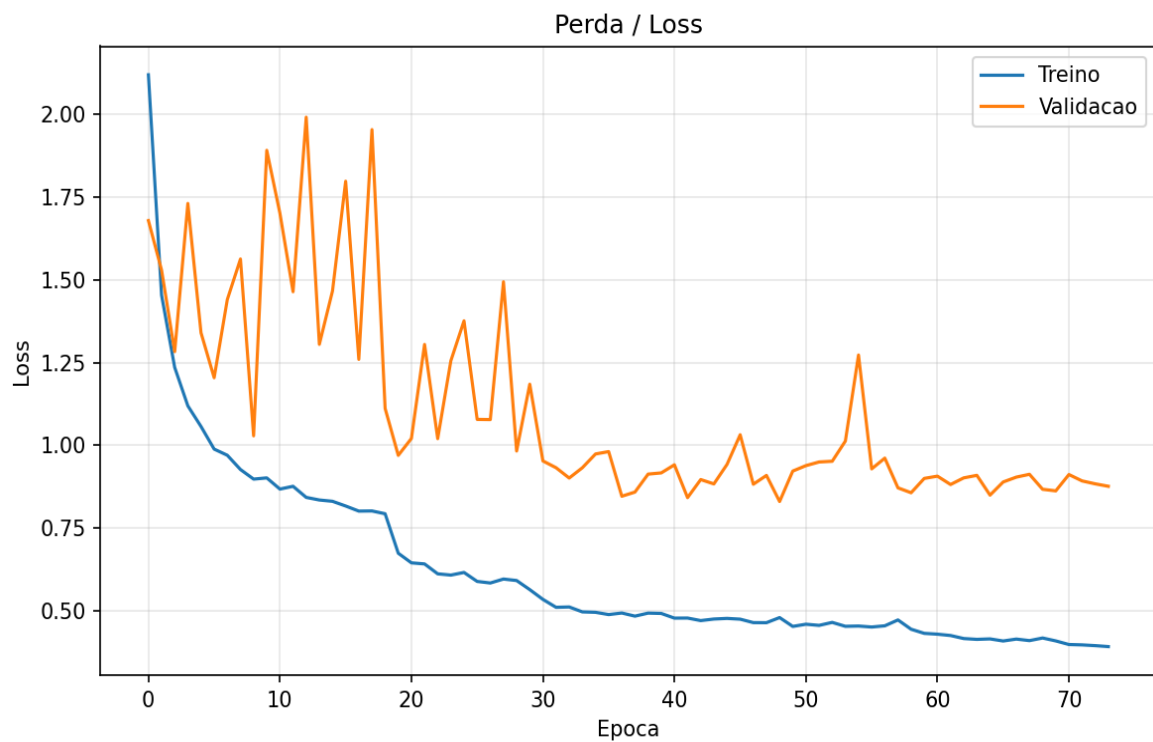
Base de vídeos de Libras referenciada no projeto: <https://libras.cin.ufpe.br/>

ANEXOS

Anexo A – Gráfico de acurácia do treinamento



Anexo B – Gráfico de perda do treinamento



Anexo C – Matriz de confusão

